

✓ BloomOS Optimized Binary Plant Health Model

Healthy vs Disease (Mobile-Ready Version)

```
!pip install kaggle -q

import os
import zipfile
import shutil
from pathlib import Path

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.utils.class_weight import compute_class_weight
from google.colab import files
```

✓ 1) Kaggle Setup

```
print("kaggle.json")
files.upload()

!mkdir -p /root/.kaggle
!mv kaggle.json /root/.kaggle/kaggle.json
!chmod 600 /root/.kaggle/kaggle.json
```

kaggle.json
 kaggle.json
kaggle.json(application/json) - 68 bytes, last modified: 10/25/2025 - 100% done
Saving kaggle.json to kaggle.json

✓ 2) Download Dataset

```
!kaggle datasets download -d csafrit2/plant-leaves-for-image-classification -p /content

zip_path = "/content/plant-leaves-for-image-classification.zip"
extract_path = "/content/plant_leaves"

if os.path.exists(extract_path):
    shutil.rmtree(extract_path)
```

```
with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)
```

Dataset URL: <https://www.kaggle.com/datasets/csafrt2/plant-leaves-for-image-classification>
License(s): Community Data License Agreement - Sharing - Version 1.0
Downloading plant-leaves-for-image-classification.zip to /content
100% 6.56G/6.56G [04:22<00:00, 26.8MB/s]

3) Create Binary Dataset: Healthy / Disease

```
SOURCE_DIR = Path(extract_path)
DATASET_DIR = Path("/content/BloomOS_dataset")
HEALTHY_DIR = DATASET_DIR / "Healthy"
DISEASE_DIR = DATASET_DIR / "Disease"

if DATASET_DIR.exists():
    shutil.rmtree(DATASET_DIR)

HEALTHY_DIR.mkdir(parents=True, exist_ok=True)
DISEASE_DIR.mkdir(parents=True, exist_ok=True)

valid_extensions = {".jpg", ".jpeg", ".png"}

for file_path in SOURCE_DIR.rglob("*"):
    if not file_path.is_file() or file_path.suffix.lower() not in valid_extensions:
        continue

    path_text = str(file_path).lower()

    # تصنيف الصور
    if "healthy" in path_text:
        target = HEALTHY_DIR
    elif "disease" in path_text or "diseased" in path_text:
        target = DISEASE_DIR
    else:
        parent_name = file_path.parent.name.lower()
        digits = "".join([c for c in parent_name if c.isdigit()])
        if digits != "":
            class_num = int(digits)
            target = HEALTHY_DIR if class_num <= 11 else DISEASE_DIR
        else:
            continue

    new_name = f"{file_path.parent.name}_{file_path.name}"
    shutil.copy2(file_path, target / new_name)

print(f"البيانات: سليمة ({len(list(HEALTHY_DIR.iterdir()))}) | مريضة ({len(list(DISEASE_DIR.iterdir()))})")

البيانات: سليمة (2273) | مريضة (2221)
```

✓ 4) Load Dataset

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 16
SEED = 42

train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
)

class_names = train_ds.class_names
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
```

```
Found 4494 files belonging to 2 classes.
Using 3596 files for training.
Found 4494 files belonging to 2 classes.
Using 898 files for validation.
```

✓ 5) Build Model (With Optimized Preprocessing)

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.1),
])

# MobileNetV2
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet",
)
base_model.trainable = False
```

```

model = keras.Sequential([
    layers.Input(shape=(224, 224, 3)),
    data_augmentation,
    # معالجة
    layers.Lambda(tf.keras.applications.mobilenet_v2.preprocess_input),
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.4),
    layers.Dense(2, activation="softmax"),
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2_1.0_224/mobilenet_v2_1.0_224_data.zip 9406464/9406464 ————— 1s 0us/step

6) Train Model

```

callbacks = [
    keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=3, restore_best_weights=True)
]

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15,
    callbacks=callbacks,
)

```

```

Epoch 1/15
225/225 ————— 419s 2s/step - accuracy: 0.7564 - loss: 0.4863 - val_accuracy: 0.7564
Epoch 2/15
225/225 ————— 496s 2s/step - accuracy: 0.8395 - loss: 0.3494 - val_accuracy: 0.8395
Epoch 3/15
225/225 ————— 423s 2s/step - accuracy: 0.8707 - loss: 0.3012 - val_accuracy: 0.8707
Epoch 4/15
225/225 ————— 436s 2s/step - accuracy: 0.8735 - loss: 0.2974 - val_accuracy: 0.8735
Epoch 5/15
225/225 ————— 422s 2s/step - accuracy: 0.8648 - loss: 0.3042 - val_accuracy: 0.8648
Epoch 6/15
225/225 ————— 425s 2s/step - accuracy: 0.8732 - loss: 0.2919 - val_accuracy: 0.8732
Epoch 7/15
225/225 ————— 459s 2s/step - accuracy: 0.8743 - loss: 0.2920 - val_accuracy: 0.8743
Epoch 8/15
225/225 ————— 500s 2s/step - accuracy: 0.8765 - loss: 0.2796 - val_accuracy: 0.8765

```



7) Save and Optimize for Mobile (TFLite)

```
model.save("plant_health_model.h5")

#TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

with open("model.tflite", "wb") as f:
    f.write(tflite_model)

with open("labels.txt", "w", encoding="utf-8") as f:
    for name in class_names:
        f.write(name + "\n")
```

```
133138959295184: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959295376: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959295568: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959293072: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959296720: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959296144: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959296336: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959296528: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959294032: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959297680: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959297104: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959297296: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959297488: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959294992: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959298640: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959299216: TensorSpec(shape=(), dtype=tf.resource, name=None)
133138959299984: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

8) Download Results

```
!zip -r BloomOS_Final_Model.zip model.tflite labels.txt plant_health_model.h5
files.download("BloomOS_Final_Model.zip")
```

```
adding: model.tflite (deflated 14%)
adding: labels.txt (stored 0%)
adding: plant_health_model.h5 (deflated 10%)
```

9) Model Performance Evaluation

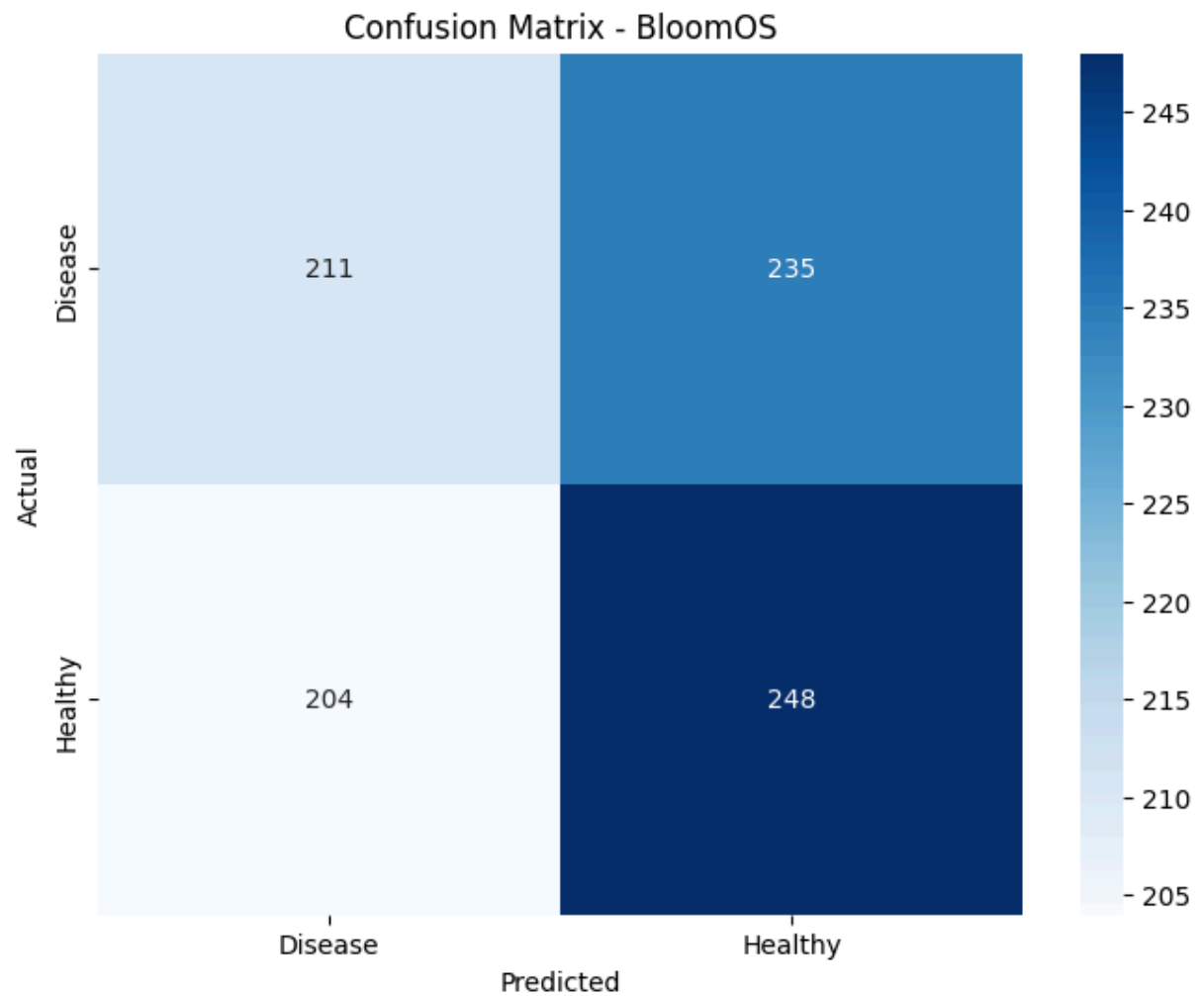
```
# metrics
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# حساب مصفوفة
cm = confusion_matrix(y_true, y_pred_classes)

# رسم المصفوفة
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabel=
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - BloomOS')
plt.show()

y_pred = model.predict(val_ds)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.concatenate([y for x, y in val_ds], axis=0)

#(F1, Precision, Recall, Accuracy)
print(classification_report(y_true, y_pred_classes, target_names=class_names))
```



57/57 ————— 88s 1s/step

	precision	recall	f1-score	support
Disease	0.49	0.46	0.47	446
Healthy	0.50	0.53	0.51	452
accuracy			0.49	898
macro avg	0.49	0.49	0.49	898
weighted avg	0.49	0.49	0.49	898

```
import matplotlib.pyplot as plt

# (history)
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(len(acc))

# رسم دقة التدريب والتحقق
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
```

```

plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.title('Training and Validation Accuracy - BloomOS')
plt.legend(loc='lower right')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.title('Training and Validation Loss - BloomOS')
plt.legend(loc='upper right')
plt.grid(True)

plt.show()

```

